Reviewer Pack — Minimal Monomial Degree vs. Resolution Proof Width Inequalit…

Entry 5db94e214b7f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 02:39:10 UTC

Minimal Monomial Degree vs. Resolution Proof Width Inequality

Entry ID: `5db94e214b7f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 02:39:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Monomial Theory) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> For every Boolean satisfiability instance φ, the minimal degree of a monomial in the Grothendieck-Witt class of its clause-indicator polynomial modulo 2 is less than or equal to twice its resolution proof width, i.e., deg(GW_φ) ≤ 2w(φ).

Rationale (proposer's reasoning):

> The use of Grothendieck-Witt classes in algebraic combinatorics provides a rich invariant for studying algebraic structures. By connecting this with the complexity of resolution proofs, we might uncover new structural insights into SAT instances that could lead to improved proof-complexity lower bounds.

Taxonomy category: `Monomial Degree vs. Resolution Proof Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `25907fa004416f61`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for at least 90% of the generated Boolean satisfiability instances φ with up to 40 variables, the Grothendieck-Witt class degree modulo 2 meets the inequality deg(GW_φ) ≤ 2w(φ), where w(φ) is the resolution proof width and deg(GW_φ) is the minimal monomial degree. The conjecture is falsified if for any instance φ, either deg(GW_φ) > 2w(φ) or the inequality does not hold for less than 90% of the instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Grothendieck-Witt degree" AND "resolution proof width" - "monomial degree" IN BOOLEAN Satisfiability - "algebraic combinatorics" AND "Boolean satisfiability"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def resolution_width(phi):
        # Simplified DPLL solver to estimate the width of the resolution proof
        stack = []
        literals = set()
        for literal in phi:
            if literal not in literals and -literal not in literals:
                literals.add(literal)
                stack.append([literal])
            else:
                if literal in literals:
                    continue
                clause = next((c for c in stack if literal in c), None)
                if clause is None:
                    return float('inf')
                stack.remove(clause)
                new_clause = [l for l in clause if l != -literal]
                if not new_clause:
                    return 0
                literals.add(literal)
                stack.append(new_clause)
        return max(len(c) for c in stack)
```

```
    def grothendieck_witt_degree(phi):
        # Simplified monomial basis method to compute the degree of the Grothendieck-Witt class
        n = int(math.log2(len(phi)))
        if n == 0:
            return 0
        degree = 1
        for i in range(1, n):
            degree *= (i + 1)
        return degree

    def deg_mod_2(gw_degree):
        return gw_degree % 2

    instances_tested = 0
    total_deg_mod_2 = 0
    n_max = 5

    for _ in range(30):
        n = random.randint(5, 40)
        phi = generate_random_boolean_instance(n)
        gw_degree = grothendieck_witt_degree(phi)
        deg_mod_2_val = deg_mod_2(gw_degree)
        width = resolution_width(phi)

        if deg_mod_2_val > 2 * width:
            return {
                "metric_name": "deg(GW_φ) mod 2",
                "metric_value": deg_mod_2_val,
                "instances_tested": instances_tested + 1,
                "n_max": n_max,
                "conjecture_holds": False,
                "counterexample": f"phi={phi}, gw_degree={gw_degree}, width={width}"
            }

        total_deg_mod_2 += deg_mod_2_val
        instances_tested += 1
        n_max = max(n_max, n)

    return {
        "metric_name": "deg(GW_φ) mod 2",
        "metric_value": total_deg_mod_2 / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = primes[:30]

    results = []
    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15047
2	propose	ollama_remote	glm4:latest	0	0	15030
3	preregistration	ollama_remote	glm4:latest	0	0	9957
4	novelty	ollama_remote	glm4:latest	0	0	8626
5	novelty	ollama_remote	glm4:latest	0	0	9654
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14933
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12548
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19058
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11655
10	judge	ollama_remote	glm4:latest	0	0	16320

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132828 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5db94e214b7f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5db94e214b7f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5db94e214b7f.tar.gz` (if generated)